

P1177 【模板】排序

题目描述

将读入的 N 个数从小到大排序后输出。

输入格式

第一行为一个正整数 N 。

第二行包含 N 个空格隔开的正整数 a_i ，为你需要进行排序的数。

输出格式

将给定的 N 个数从小到大输出，数之间空格隔开，行末换行且无空格。

输入输出样例 #1

输入 #1

```
5
4 2 4 5 1
```

输出 #1

```
1 2 4 4 5
```

说明/提示

对于 20% 的数据，有 $1 \leq N \leq 10^3$ ；

对于 100% 的数据，有 $1 \leq N \leq 10^5$ ， $1 \leq a_i \leq 10^9$ 。

题解

题目分析

- 题目要求：**给定 N 个正整数，将其从小到大排序后输出，要求行末换行且无空格。
- 数据规模：**
 - 20%数据： $1 \leq N \leq 10^3$ ，简单排序算法即可满足需求。
 - 100%数据： $1 \leq N \leq 10^5$ ， $1 \leq a_i \leq 10^9$ ，此时需采用时间复杂度更优的排序算法，归并排序便是合适选择，其时间复杂度为 $O(N \log N)$ ，能高效处理大规模数据。

算法原理：归并排序

归并排序基于**分治思想**，核心流程分为“分”和“合”两步：

1. **分 (Divide)**：将待排序数组不断从中点分割，直到每个子数组只包含1个元素（单个元素可视为已排序）。例如，数组[4, 2, 4, 5, 1]会先分割为[4, 2, 4]和[5, 1]，再继续分割为[4, 2]、[4]、[5]、[1]，最终分割为[4]、[2]、[4]、[5]、[1]。
2. **合 (Merge)**：将两个已排序的子数组合并为一个有序数组。合并过程需借助临时数组，通过双指针分别遍历两个子数组，依次将较小的元素放入临时数组，最后将临时数组的元素拷贝回原数组，完成排序。

代码解析

1. 分治排序函数 (merge_sort)

```
void merge_sort(int left, int right) {  
    if (left >= right) return; // 递归终止条件：子数组长度为1  
  
    int mid = (left + right) / 2; // 分割数组为左右两部分  
  
    merge_sort(left, mid); // 递归排序左子数组  
    merge_sort(mid + 1, right); // 递归排序右子数组  
  
    merge(left, right); // 合并左右两个已排序子数组  
}
```

- **功能**：通过递归实现“分”的过程，先排序左右子数组，再调用 `merge` 函数完成“合”的过程，最终使 `arr[left..right]` 有序。
- **递归终止条件**：当 `left >= right` 时，子数组仅含1个元素，无需继续分割和排序。

2. 合并函数 (merge)

```
void merge(int left, int right) {  
    if (left >= right) return; // 子数组只有1个元素，无需合并  
    int mid = (left + right) / 2; // 找到当前数组的中点  
    int i = left, j = mid + 1, k = 0; // i: 左子数组指针; j: 右子数组指针; k: 临时数组指针  
  
    // 双指针遍历两个子数组，将较小元素放入临时数组  
    while (i <= mid && j <= right) {  
        if (arr[i] <= arr[j]) {  
            temp[k++] = arr[i++];  
        } else {  
            temp[k++] = arr[j++];  
        }  
    }  
  
    // 将左子数组剩余元素拷贝到临时数组  
    while (i <= mid) temp[k++] = arr[i++];  
    // 将右子数组剩余元素拷贝到临时数组  
    while (j <= right) temp[k++] = arr[j++];  
  
    // 将临时数组的有序数据拷贝回原数组arr[left..right]  
    for (int p = 0; p < k; p++) {  
        arr[left + p] = temp[p];  
    }  
}
```

```
}
```

- **功能**: 将 `arr[left..mid]` 和 `arr[mid+1..right]` 两个已排序子数组合并为 `arr[left..right]` 有序数组。
- **关键逻辑**: 双指针确保每次取两个子数组中最小的元素, 保证合并后数组有序; 剩余元素拷贝步骤确保无元素遗漏。

算法复杂度分析

- **时间复杂度**: $O(N \log N)$
 - “分”的过程: 将数组分割为 N 个单个元素, 需 $\log N$ 层递归 (如 $N = 5$ 时, 递归层数为 3)。
 - “合”的过程: 每一层合并的总元素个数均为 N , 共 $\log N$ 层, 因此总时间为 $O(N \log N)$ 。
- **空间复杂度**: $O(N)$
 - 需额外使用大小为 N 的临时数组 `temp`, 递归栈空间为 $O(\log N)$ (可忽略), 因此总体空间复杂度为 $O(N)$ 。

P1226 【模板】快速幂

题目描述

给你三个整数 a, b, p , 求 $a^b \bmod p$ 。

输入格式

输入只有一行三个整数, 分别代表 a, b, p 。

输出格式

输出一行一个字符串 `a^b mod p=s`, 其中 a, b, p 分别为题目给定的值, s 为运算结果。

输入输出样例 #1

输入 #1

```
2 10 9
```

输出 #1

```
2^10 mod 9=7
```

说明/提示

样例解释

$2^{10} = 1024$, $1024 \bmod 9 = 7$ 。

数据规模与约定

对于 100% 的数据, 保证 $0 \leq a, b < 2^{31}$, $a + b > 0$, $2 \leq p < 2^{31}$ 。

题解

题目分析

本题要求计算 $a^b \bmod p$ 的结果，其中 a, b 可以是接近 2^{31} 的大数。直接计算 a^b 会导致数值溢出，且当 b 极大时，普通的循环乘法效率极低，因此需要使用快速幂算法来高效求解。

题目特点：

- 输入范围大： $0 \leq a, b < 2^{31}$, $2 \leq p < 2^{31}$
- 需处理模运算：结果要对 p 取模

算法原理

快速幂算法（二进制 exponentiation）的核心思想是利用指数的二进制表示，将幂运算转化为平方和乘法的组合，从而大幅减少计算次数。

基本原理：

1. 当 b 为偶数时： $a^b = (a^{b/2})^2$
2. 当 b 为奇数时： $a^b = (a^{b/2})^2 \times a$
3. 递归终止条件：当 $b = 0$ 时， $a^0 = 1$

同时利用模运算的性质： $(x \times y) \bmod p = [(x \bmod p) \times (y \bmod p)] \bmod p$ ，在**每一步计算中都进行取模操作**，避免数值溢出。

代码分析

- 递归写法

```
#define ll long long
ll qpow(ll a, int b, ll p){
    if (b==0) return 1;
    ll res=qpow(a,b/2,p);
    ll ans=1;
    if (b%2==1) ans=((res*res)%p)*a%p;
    else ans=res*res%p;
    return ans;
}
```

- 循环写法

```
while(b>0){ //如果b还不是0
    if(b&1) ans=ans*a%p; //如果最低位是1 就把ans*a
    a=a*a%p; //不论最低位是不是1, a都需要自乘以升幂
    b>>=1; //b右移一位
}
```

算法复杂度分析

- **时间复杂度**： $O(\log b)$ 。因为每一步递归都将指数 b 减半，递归深度为 b 的二进制位数，约为 $\log_2 b$ ，每一层递归仅进行常数次运算。
- **空间复杂度**： $O(\log b)$ 。由于使用递归实现，递归调用栈的深度为 $\log_2 b$ 。

相比朴素的循环乘法（时间复杂度 $O(b)$ ），**快速幂算法在处理大指数时效率提升极为显著**，尤其适合本题中 b 接近 2^{31} 的情况。

P1908 逆序对

题目描述

猫猫 TOM 和小老鼠 JERRY 最近又较量上了，但是毕竟都是成年人，他们已经不喜欢再玩那种你追我赶的游戏，现在他们喜欢玩统计。

最近，TOM 老猫查阅到一个人类称之为“逆序对”的东西，这东西是这样定义的：对于给定的一段正整数序列，逆序对就是序列中 $a_i > a_j$ 且 $i < j$ 的有序对。知道这概念后，他们就比赛谁先算出给定的一段正整数序列中逆序对的数目。注意序列中可能有重复数字。

Update：数据已加强。

输入格式

第一行，一个数 n ，表示序列中有 n 个数。

第二行 n 个数，表示给定的序列。序列中每个数字不超过 10^9 。

输出格式

输出序列中逆序对的数目。

输入输出样例 #1

输入 #1

```
6
5 4 2 6 3 1
```

输出 #1

```
11
```

说明/提示

对于 25% 的数据， $n \leq 2500$ 。

对于 50% 的数据， $n \leq 4 \times 10^4$ 。

对于所有数据， $1 \leq n \leq 5 \times 10^5$ 。

请使用较快的输入输出。

题解

题目分析

逆序对问题是一个经典的算法问题，其定义为：对于一个序列中的两个元素 a_i 和 a_j ，如果 $i < j$ 且 $a_i > a_j$ ，则这两个元素构成一个逆序对。题目要求统计给定序列中所有逆序对的数量。

从输入输出样例可以看出，对于序列 `5 4 2 6 3 1`，其逆序对总数为 11 对。

题目数据规模要求我们处理 $n \leq 5 \times 10^5$ 的情况，这意味着我们需要一个时间复杂度优于 $O(n^2)$ 的算法，否则会出现超时。

算法原理

解决逆序对问题的高效算法通常是基于归并排序的思想，其核心原理是：

1. 采用分治法，将序列不断二分，直到每个子序列只有一个元素
2. 在合并两个已排序子序列的过程中，统计逆序对的数量
3. 当左边子序列的元素大于右边子序列的元素时，左边子序列中当前元素及之后的所有元素都与右边子序列的当前元素构成逆序对

这种方法利用了归并排序的特性，在排序的同时完成逆序对的统计，从而达到 $O(n \log n)$ 的时间复杂度。

随便举个例子：

3 2 4 6 7 1 5 8

3 2 4 6 | 7 1 5 8

3 2 | 4 6 | 7 1 | 5 8

3 | 2 | 4 | 6 | 7 | 1 | 5 | 8

2 3 | 4 6 | 1 7 | 5 8

1 || 0 || 1 || 0 —— 2

2 3 4 6 | 1 5 7 8

0 || 1 —— 1

2 3 4 6

1 5 7 8

$4 + 1 + 0 + 0 = 5$ —— 5

$\text{ans} = 2 + 1 + 5 = 8$

代码分析

```
void msort(int b,int c){
    if(b == c)return;
    int mid = (b + c) / 2;
    int i = b, j = mid + 1, k = b;
    msort(b, mid);
    msort(mid + 1, c);
    while(i <= mid && j <= c){
        if(a[i] <= a[j])
            d[k++] = a[i++];
        else
            d[k++] = a[j++], ans = ans + mid - i + 1;
    }
    while(i <= mid) d[k++] = a[i++];
    while(j <= c) d[k++] = a[j++];
    for(int l = b ; l <= c ; l++)
        a[l] = d[l];
}
```

代码关键部分解析：

1. 全局变量 `ans` 用于累计逆序对的数量，使用 `long long` 类型是为了防止在数据量大时发生溢出。
2. `msort` 函数实现了归并排序并统计逆序对：
 - 先递归地将数组二分，直到子数组长度为1。
 - 在合并两个有序子数组时，当发现左子数组元素大于右子数组元素时，说明存在逆序对。
 - 此时逆序对的数量为左子数组中剩余元素的个数，即 `mid - i + 1`。
3. 临时数组 `d` 用于存储合并过程中的中间结果，最后会将排序好的结果复制回原数组 `a`。

算法复杂度分析

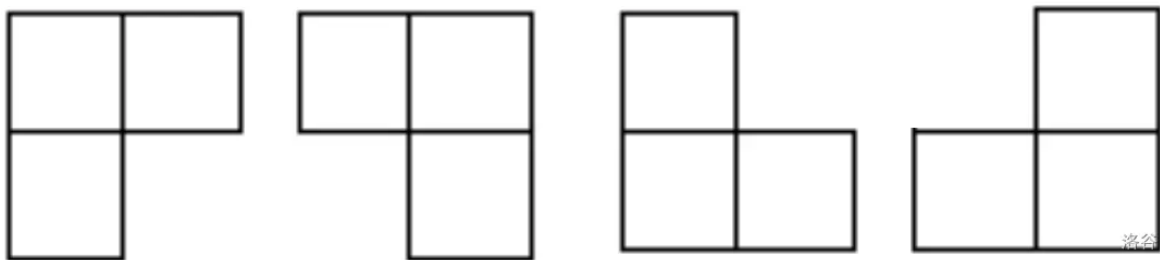
- **时间复杂度：** $O(n \log n)$
 - 归并排序的二分过程形成了深度为 $\log n$ 的递归树
 - 每一层的合并操作总共需要 $O(n)$ 的时间
 - 因此总体时间复杂度为 $O(n \log n)$ ，能够处理 $n \leq 5 \times 10^5$ 的数据规模
- **空间复杂度：** $O(n)$
 - 需要一个与原数组大小相同的临时数组 `d` 来存储合并结果
 - 递归调用的栈空间为 $O(\log n)$ ，可以忽略不计
 - 因此总体空间复杂度为 $O(n)$

这种基于归并排序的方法是解决逆序对问题的经典高效解法，相比暴力枚举的 $O(n^2)$ 算法，大大提高了处理大规模数据的能力。

B2163 棋盘覆盖

题目描述

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其他方格不同，称该方格为一特殊方格。现在用 L 型（占 3 小格）骨牌覆盖棋盘上除了特殊方格的所有方格，各骨牌不能重叠。



现给出棋盘的大小和特殊方格所在的位置，请找出这种棋盘。

输入格式

输入两行。

第一行一个正整数 k 。

第二行两个正整数 x, y ，表示第 x 行第 y 列为特殊方格。

输出格式

输出一个 2^k 行 2^k 列的棋盘，每两个整数之间由空格隔开。特殊方格用 0 表示。对于其他方格，从 1 开始编号骨牌，输出该方格所属的骨牌编号。

输出任意一种符合要求的答案都算作正确。

输入输出样例 #1

输入 #1

```
2
2 2
```

输出 #1

```
2 2 3 3
2 0 1 3
4 1 1 5
4 4 5 5
```

说明/提示

本题共有 10 个子任务，第 i 个子任务中 $k = i$ 。每个子任务记 10 分，你只有通过每个子任务中所有测试点才能获得该子任务的分数。

题解

题目分析

1. 问题描述

在一个 $2^k \times 2^k$ 的棋盘里，存在一个特殊方格（用 0 表示）。要求使用 L 型骨牌（每个骨牌覆盖 3 个方格）覆盖棋盘上除特殊方格之外的所有方格，且骨牌不能重叠。最后输出覆盖结果，非特殊方格需标注其所属骨牌的编号（从 1 开始）。

2. 输入输出要求

- 输入：**第一行是正整数 k ，用于确定棋盘大小为 $2^k \times 2^k$ ；第二行是两个正整数 x 和 y ，代表特殊方格所在的行和列（行和列均从 1 开始计数）。
- 输出：**输出一个 2^k 行 2^k 列的矩阵，矩阵元素间用空格隔开，特殊方格为 0，其余方格为所属骨牌编号。

3. 样例分析

以输入样例“2 2 2”为例，棋盘大小为 $2^2 \times 2^2 = 4 \times 4$ ，特殊方格在第 2 行第 2 列。输出矩阵中，0 位于第 2 行第 2 列，其他位置的数字代表所属骨牌编号，如第 1 行前两个方格都为 2，表明它们属于 2 号骨牌，且骨牌覆盖符合 L 型要求，无重叠且覆盖了除特殊方格外的所有区域。

算法原理

本题采用**分治法**来解决棋盘覆盖问题，分治法的核心思想是将一个复杂问题分解为多个规模较小且与原问题结构相似的子问题，递归解决子问题后，再将子问题的解合并得到原问题的解。具体步骤如下：

- 问题分解：**将 $2^k \times 2^k$ 的棋盘沿着中线分为四个 $2^{k-1} \times 2^{k-1}$ 的子棋盘。此时，特殊方格必然位于其中一个子棋盘内。
- 构造子问题：**对于没有特殊方格的三个子棋盘，在它们靠近原棋盘中心的位置各放置一个与当前骨牌编号相同的方格（这三个方格与特殊方格所在子棋盘靠近中心的方格共同构成一个 L 型骨牌），将这三个方格视为对应子棋盘的“特殊方格”，从而使每个子棋盘都转化为一个包含“特殊方格”的小规模棋盘覆盖问题。

3. **递归求解**：对四个子棋盘分别递归执行上述分解和构造操作，直到子棋盘的大小为 1×1 （此时无需覆盖，递归终止）。
4. **骨牌编号**：使用一个全局计数器，每处理一个 L 型骨牌（即每构造一次三个子棋盘的“特殊方格”），计数器值加 1，用于标注骨牌编号。

举个例子：

输入

```
3
5 6
```

输出

```
3 3 4 4 8 8 9 9
3 2 2 4 8 7 7 9
5 2 6 6 10 10 7 11
5 5 6 1 1 10 11 11
13 13 14 1 18 0 19 19
13 12 14 14 18 18 17 19
15 12 12 16 20 17 17 21
15 15 16 16 20 20 21 21
```

为了看得更直观，我们简单对齐一下：

```
3  3  4  4  8  8  9  9
3  2  2  4  8  7  7  9
5  2  6  6 10 10  7 11
5  5  6  1  1 10 11 11
13 13 14 1 18 0 19 19
13 12 14 14 18 18 17 19
15 12 12 16 20 17 17 21
15 15 16 16 20 20 21 21
```

代码分析

1. 全局变量与数组定义

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const int MAX_N = 1200; // 最大棋盘尺寸， $2^{10}=1024$ ，故1200足够
int a[MAX_N][MAX_N];    // 存储棋盘覆盖结果
int cnt = 1;             // 骨牌编号计数器，从1开始
```

- `MAX_N`：考虑到 k 最大可能为 10 ($2^{10} = 1024$)，设置为 1200 可满足所有可能的棋盘大小需求。
- `a` 二维数组：用于记录每个方格所属的骨牌编号或特殊方格 (0)。
- `cnt`：全局计数器，保证骨牌编号的唯一性和连续性。

2. 递归函数 chessboard

```
void chessboard(int x, int y, int sx, int sy, int sz){
    if(sz == 1) return ; // 子棋盘大小为1x1, 无需覆盖, 递归终止
    int s = sz / 2;      // 子棋盘的边长
    int num = cnt++;      // 当前使用的骨牌编号, 计数器自增

    // 处理左上角子棋盘 (x~x+s-1, y~y+s-1)
    if (sx < x + s && sy < y + s) // 特殊方格在此子棋盘内
        chessboard(x, y, sx, sy, s);
    else { // 特殊方格不在此子棋盘, 将子棋盘右下角设为当前骨牌编号 (作为“特殊方格”)
        a[x+s-1][y+s-1] = num;
        chessboard(x, y, x + s - 1, y + s - 1, s);
    }

    // 处理右上角子棋盘 (x~x+s-1, y+s~y+sz-1)
    if (sx < x + s && sy >= y + s) // 特殊方格在此子棋盘内
        chessboard(x, y + s, sx, sy, s);
    else { // 特殊方格不在此子棋盘, 将子棋盘左下角设为当前骨牌编号 (作为“特殊方格”)
        a[x+s-1][y+s] = num;
        chessboard(x, y + s, x + s - 1, y + s, s);
    }

    // 处理左下角子棋盘 (x+s~x+sz-1, y~y+s-1)
    if (sx >= x + s && sy < y + s) // 特殊方格在此子棋盘内
        chessboard(x + s, y, sx, sy, s);
    else { // 特殊方格不在此子棋盘, 将子棋盘右上角设为当前骨牌编号 (作为“特殊方格”)
        a[x+s][y+s-1] = num;
        chessboard(x + s, y, x + s, y + s - 1, s);
    }

    // 处理右下角子棋盘 (x+s~x+sz-1, y+s~y+sz-1)
    if (sx >= x + s && sy >= y + s) // 特殊方格在此子棋盘内
        chessboard(x + s, y + s, sx, sy, s);
    else { // 特殊方格不在此子棋盘, 将子棋盘左上角设为当前骨牌编号 (作为“特殊方格”)
        a[x+s][y+s] = num;
        chessboard(x + s, y + s, x + s, y + s, s);
    }
}
```

- 函数参数:

- `x, y`: 当前子棋盘的左上角坐标;
- `sx, sy`: 特殊方格的坐标;
- `sz`: 当前子棋盘的边长 (始终为 2 的幂)。

- 递归终止条件: 当 `sz == 1` 时, 子棋盘只有一个方格, 若为特殊方格则已标记为 0, 否则无需处理, 直接返回。

- 子棋盘处理逻辑:

1. 计算子棋盘边长 `s = sz / 2`, 获取当前骨牌编号 `num` 并更新计数器。
2. 依次判断特殊方格是否在四个子棋盘中:
 - 若在, 则递归处理该子棋盘;
 - 若不在, 则在子棋盘靠近原棋盘中心的位置标记当前骨牌编号 (作为该子棋盘的“特殊方格”), 再递归处理。

3. 主函数

```
int main(){
    int k, sx, sy;
    cin >> k >> sx >> sy;           // 输入k和特殊方格坐标
    int sz = 1 << k;                  // 计算棋盘边长, 1<<k等价于2^k
    chessboard(1, 1, sx, sy, sz);     // 从整个棋盘(左上角(1,1))开始递归覆盖
    // 输出棋盘覆盖结果
    for (int i = 1; i <= sz; i++){
        for (int j = 1; j <= sz; j++){
            cout << a[i][j] << ' ';
        }
        cout<<endl;
    }
    return 0;
}
```

- 首先读取输入的 k 、特殊方格的行号 sx 和列号 sy 。
- 通过 $1 \ll k$ 计算棋盘边长（利用位运算提高效率，等价于 2^k ）。
- 调用 `chessboard` 函数，从整个棋盘的左上角 (1,1) 开始递归覆盖。
- 最后通过双重循环遍历二维数组 `a`，按行输出棋盘的覆盖结果。

算法复杂度分析

1. 时间复杂度

设棋盘的边长为 $n = 2^k$ ，算法的时间复杂度可通过递归关系式分析：

- 递归终止条件：当 $n = 1$ 时，时间复杂度为 $O(1)$ （直接返回）。
- 递归递推关系：当 $n > 1$ 时，将原问题分解为 4 个规模为 $n/2$ 的子问题，每个子问题的处理时间为 $T(n/2)$ ；同时，在每个递归层次中，除了递归调用外的其他操作（如判断特殊方格位置、标记骨牌编号）的时间复杂度为 $O(1)$ 。因此，递归关系式为：

$$T(n) = 4T(n/2) + O(1)$$

[重谈主定理 \(master定理\) 及其证明](#)

通过**主定理**求解该递归关系式：

主定理适用于形式为 $T(n) = aT(n/b) + f(n)$ 的递归式，其中 $a \geq 1$, $b > 1$, $f(n)$ 是渐近正函数。本题中， $a = 4$, $b = 2$, $f(n) = O(1) = O(n^0)$ 。由于 $\log_b a = \log_2 4 = 2 > 0$ ，满足主定理的第一种情况，即 $f(n) = O(n^{\log_b a - \epsilon})$ （其中 $\epsilon = 2$ ），因此：

$$T(n) = O(n^{\log_b a}) = O(n^2)$$

由于 $n = 2^k$ ，则 $n^2 = 4^k$ ，故时间复杂度也可表示为 $O(4^k)$ ，即与棋盘的方格总数成正比，效率较高。

2. 空间复杂度

算法的空间复杂度主要由两部分构成：

- 存储棋盘覆盖结果的二维数组 `a`：其大小为 $n \times n = 2^k \times 2^k = 4^k$ ，因此空间复杂度为 $O(4^k)$ 。
- 递归调用栈的空间：递归的深度等于将棋盘从 n 分解到 1 的次数，即 k 次（因为每次分解后边长减半， $2^k \rightarrow 2^{k-1} \rightarrow \dots \rightarrow 2^0 = 1$ ），因此递归栈的空间复杂度为 $O(k)$ 。

由于 4^k 远大于 $O(k)$ （当 $k \geq 1$ 时），因此算法的整体空间复杂度为 $O(4^k)$ （或 $O(n^2)$ ）。

P5461 赦免战俘

题目背景

借助反作弊系统，一些在月赛有抄袭作弊行为的选手被抓出来了！

题目描述

现有 $2^n \times 2^n$ ($n \leq 10$) 名作弊者站成一个正方形方阵等候 kkksc03 的发落。kkksc03 决定赦免一些作弊者。他将正方形矩阵均分为 4 个更小的正方形矩阵，每个更小的矩阵的边长是原矩阵的一半。其中左上角那一个矩阵的所有作弊者都将得到赦免，剩下 3 个小矩阵中，每一个矩阵继续分为 4 个更小的矩阵，然后通过同样的方式赦免作弊者……直到矩阵无法再分下去为止。所有没有被赦免的作弊者都将被处以棕名处罚。

给出 n ，请输出每名作弊者的命运，其中 0 代表被赦免，1 代表不被赦免。

输入格式

一个整数 n 。

输出格式

$2^n \times 2^n$ 的 01 矩阵，代表每个人是否被赦免。数字之间有一个空格。

输入输出样例 #1

输入 #1

```
3
```

输出 #1

```
0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 1
0 0 0 0 0 1 0 1
0 0 0 0 1 1 1 1
0 0 0 1 0 0 0 1
0 0 1 1 0 0 1 1
0 1 0 1 0 1 0 1
1 1 1 1 1 1 1 1
```

题解

题目分析

本题要求我们生成一个 $2^n \times 2^n$ 的 01 矩阵，模拟赦免战俘的过程。赦免规则是：将正方形矩阵均分为 4 个更小的正方形矩阵，赦免左上角的矩阵，然后对剩下的 3 个矩阵递归执行同样的操作，直到矩阵无法再分（即大小为 1×1 ）。最终矩阵中 0 表示被赦免，1 表示未被赦免。

例如当 $n=3$ 时，我们需要生成 8×8 的矩阵，左上角 4×4 的矩阵被赦免，然后对右上角、左下角和右下角的 4×4 矩阵递归执行操作。

算法原理

解决本题目的核心思想是递归分治：

1. 对于给定大小的矩阵，将其分为4个大小相等的子矩阵；
2. 左上角的子矩阵全部赦免（保持0）；
3. 对其余3个子矩阵（右上角、左下角、右下角）递归执行相同的操作；
4. 递归终止条件是矩阵大小为 2×2 ，此时我们手动设置右下角为1，其他三个位置中左上角保持0，其余两个设为1。

这种递归方式能够精确地按照题目要求的规则填充矩阵，确保每个应该被赦免的位置都是0，其余位置都是1。

代码分析

```
#include<bits/stdc++.h>
using namespace std;
const int MAX_N = 1200;
int a[MAX_N][MAX_N];
void dfs(int x, int y, int sz){
    if (sz == 2){
        a[x+1][y+1] = 1;
        a[x][y+1] = 1;
        a[x+1][y] = 1;
        return ;
    }
    int s = sz / 2;
    dfs(x + s, y, s);
    dfs(x, y + s, s);
    dfs(x + s, y + s, s);
}
int main(){
    int n;
    cin >> n;
    n = 1 << n;
    dfs(1, 1, n);
    for (int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            cout << a[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

代码的执行流程：

1. 首先定义了一个足够大的二维数组 `a` 来存储结果矩阵；
2. `dfs` 函数是核心，它接收三个参数：子矩阵左上角坐标(x,y)和子矩阵大小sz；
3. 当sz=2时，到达递归终止条件，手动设置三个位置为1；
4. 对于更大的矩阵，将其分为四个子矩阵，递归处理除左上角外的三个子矩阵；
5. 主函数中计算矩阵大小为 2^n ，调用dfs处理整个矩阵，最后输出结果。

值得注意的是，数组默认初始化为0，所以被赦免的位置（左上角子矩阵）不需要额外处理，保持0即可。

算法复杂度分析

- **时间复杂度**: $O(4^n)$, 也可以表示为 $O(N^2)$, 其中 $N = 2^n$ 是矩阵的边长。因为输出的时候, 每个位置都需要被访问一次, 总共有 N^2 个位置。
- **空间复杂度**: $O(N^2)$, 主要是存储结果矩阵的二维数组占用的空间。递归调用栈的深度为 n , 因为每次递归矩阵大小减半, 从 2^n 减小到 2 需要 n 步, 所以递归栈的空间复杂度是 $O(n)$, 可以忽略不计。

P6687 论如何玩转 Excel 表格

题目描述

有一个 $2 \times n$ 的表格, 表格内不重不漏地填有 $1 \sim 2 \times n$ 这些数字。你可以进行若干次操作, 每次操作可以选择一个 2×2 的正方形区域, 然后旋转 180° 。

给出现在的状态以及目标状态。问是否可以达到目标状态。如果能, 最少操作次数是多少。

输入格式

第一行一个正整数 n 。

接下来两行, 每行 n 个正整数表示表格现在的状态。

接下来两行, 每行 n 个正整数表示目标状态。

输出格式

若无解, 输出一行 `d1dsgay!!1`。

否则, 输出一个整数, 表示最少的操作次数。

输入输出样例 #1

输入 #1

```
1
1
2
2
1
```

输出 #1

```
d1dsgay!!1
```

输入输出样例 #2

输入 #2

```
2
1 2
3 4
4 3
2 1
```

输出 #2

```
1
```

输入输出样例 #3

输入 #3

```
2
1 2
3 4
1 2
4 3
```

输出 #3

```
d1dsgay!!1
```

输入输出样例 #4

输入 #4

```
3
1 6 2
4 3 5
2 4 3
5 1 6
```

输出 #4

```
2
```

说明/提示

子任务

本题采用捆绑测试。

对于 100% 的数据： $1 \leq n \leq 10^6$ 。

本题共 10 个子任务，各子任务的分值和限制如下：

- 子任务 1 (1 分) : 保证输出一定为 `d1dsgay!!1`。
- 子任务 2 (1 分) : 保证目标状态与现在状态一致。
- 子任务 3 (4 分) : $n = 1$ 。
- 子任务 4 (5 分) : $n = 2$ 。
- 子任务 5 (14 分) : $1 \leq n \leq 20$ 。
- 子任务 6 (19 分) : $1 \leq n \leq 100$ 。
- 子任务 7 (19 分) : $1 \leq n \leq 1000$ 。
- 子任务 8 (8 分) : 保证目标状态第一行依次为 $1 \sim n$, 第二行依次为 $n + 1 \sim 2n$ 。
- 子任务 9 (10 分) : 保证不存在无解情况。
- 子任务 10 (19 分) : 无特殊限制。

题解

此题卡输入输出!!! (对于这种输入比较大的题目, 建议养成关流的良好习惯: -)

```
std::ios::sync_with_stdio(false);
std::cin.tie(nullptr);
```

题目分析

本题要求我们判断一个 $2 \times n$ 的表格是否可以通过若干次 2×2 区域的 180° 旋转操作, 从初始状态转变为目标状态。如果可以, 还要求出最少的操作次数。

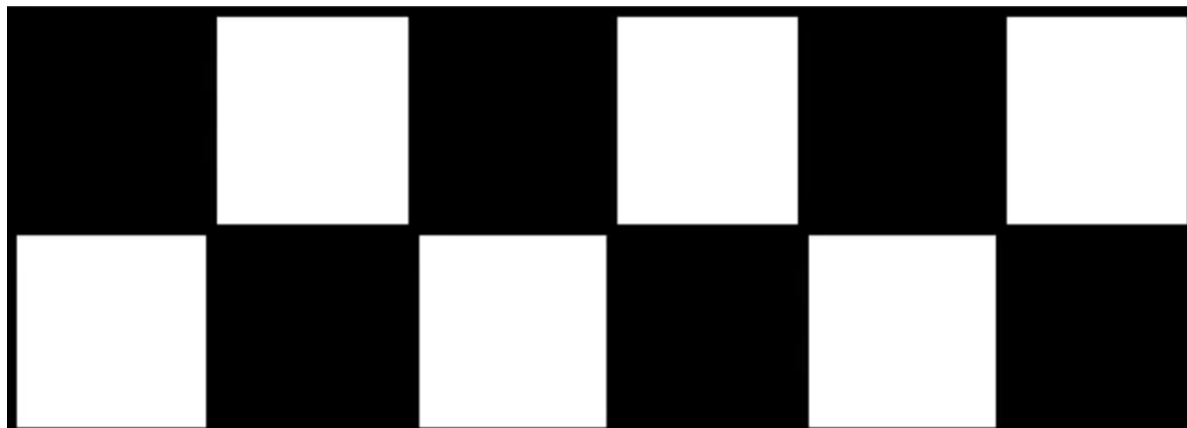
每次操作是选择一个 2×2 的正方形区域进行 180° 旋转, 这会导致该区域内的四个元素按顺时针方向互换位置。例如:

```
a b    →    d c
c d        b a
```

算法原理

解决这道题的关键在于发现旋转操作的特性和限制:

- 垂直配对守恒**: 每一列的两个数字必须始终保持配对关系。即如果初始状态中 a 和 b 在同一列, 那么任何操作都不能将它们分开, 它们必须始终在同一列 (可能上下位置互换)。这是第一个判断无解的条件。
- 位置奇偶性守恒**: 每个数字所在位置的 $(i+j)$ 奇偶性是不变的。即如果一个数字初始在 $(i+j)$ 为偶数的位置, 那么它永远只能出现在 $(i+j)$ 为偶数的位置。这是第二个判断无解的条件。
- 最少操作次数计算**: 当确认问题有解后, 我们需要计算最少操作次数。通过分析可以发现, 这个问题可以转化为计算逆序对的数量。具体来说, 我们将目标状态中每列第一个元素在初始状态中的列索引构成一个数组, 该数组的逆序对数量就是最少操作次数。



代码分析

```
#include<bits/stdc++.h>
using namespace std;
const int MAX_N = 1e6 + 6;
const int MAX_NUM = 2e6 + 6;
int a[3][MAX_N], b[3][MAX_N]; // 存储初始状态和目标状态
int nx[MAX_NUM], c[MAX_NUM], d[MAX_NUM]; // nx存储配对数字, c存储位置奇偶性, d存储所在列
int arr[MAX_N], tmp[MAX_N]; // 用于计算逆序对
long long ans; // 存储最少操作次数

// 归并排序计算逆序对
void msort(int x, int y){
    if(x == y)return;
    int mid = (x + y) / 2;
    int i = x, j = mid + 1, k = x;
    msort(x, mid);
    msort(mid + 1, y);
    while(i <= mid && j <= y){
        if(arr[i] <= arr[j])
            tmp[k++] = arr[i++];
        else
            tmp[k++] = arr[j++], ans = ans + mid - i + 1; // 统计逆序对
    }
    while(i <= mid) tmp[k++] = arr[i++];
    while(j <= y) tmp[k++] = arr[j++];
    for(int l = x; l <= y; l++)
        arr[l] = tmp[l];
}

int main(){
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    bool flag = false;
    scanf("%d", &n);

    // 读取初始状态并记录相关信息
    for (int i = 1; i <= 2; i++){
        for (int j = 1; j <= n; j++){
            scanf("%d", &a[i][j]);
            c[a[i][j]] = (i + j) % 2; // 记录位置奇偶性
        }
    }

    // 记录配对关系和所在列
    for (int j = 1; j <= n; j++){
        nx[a[1][j]] = a[2][j]; // 记录配对数字
        nx[a[2][j]] = a[1][j];
        d[a[1][j]] = j; // 记录所在列
        d[a[2][j]] = j;
    }

    // 读取目标状态并检查奇偶性是否匹配
    for (int i = 1; i <= 2; i++){
        for (int j = 1; j <= n; j++){
```

```

        scanf("%d", &b[i][j]);
        if (c[b[i][j]] != (i + j) % 2) flag = true; // 奇偶性不匹配, 无解
    }
}

// 检查配对关系是否保持
for (int j = 1; j <= n; j++){
    if (nx[b[1][j]] != b[2][j]) { // 配对关系被破坏, 无解
        printf("d!dsgay!!1\n");
        return 0;
    }
}

// 特殊情况处理: n=1
if (n == 1 && a[1][1] == b[1][1]){
    printf("0\n");
    return 0;
}
if (n == 1 || flag) { // n=1且不匹配, 或奇偶性不匹配
    printf("d!dsgay!!1\n");
    return 0;
}

// 构建数组并计算逆序对
for(int j = 1; j <= n; j++){
    arr[j] = d[b[1][j]]; // 目标状态第一行元素在初始状态中的列索引
}
msort(1, n); // 归并排序计算逆序对
printf("%lld\n", ans);
return 0;
}

```

算法复杂度分析

1. 时间复杂度:

- 读取输入并进行初步检查的时间复杂度为 $O(n)$ 。
- 使用归并排序计算逆序对的时间复杂度为 $O(n\log n)$ 。
- 整体时间复杂度为 $O(n\log n)$, 能够处理 $n = 1e6$ 的情况。

2. 空间复杂度:

- 使用了多个数组存储状态信息, 空间复杂度为 $O(n)$ 。
- 归并排序递归调用的栈空间为 $O(\log n)$ 。
- 整体空间复杂度为 $O(n)$ 。